

Databases Project Part 4

1. Review feedback provided for PART 3 and make necessary changes.

ERD(No Changes)

Relational Schema(No changes)

RA Changes:

Old E:

$\text{Payments_CY} \leftarrow \sigma (\text{Today} - \text{BillingRecord.Date} \leq 365) \text{ BillingRecord}$

$\text{Patients_Payments_CY} \leftarrow \text{Payments_CY} \bowtie \text{Payments_CY.SSN} = \text{Patients.SSN} \text{ Patients}$

$\text{Patient_Totals} \leftarrow \text{Patients_Payments_CY} \bowtie \text{Payments_CY.SSN} = \text{Patient.SSN} \Pi$

$\text{SUM}(\text{PaymentAmount}) (\text{Patients_Payments_CY}) \text{ Top_Patient} \leftarrow \sigma \text{ Total} =$

$\text{MAX}(\text{Patient_Totals.Total}) \text{ Top_Person} \leftarrow \text{Top_Patient} \bowtie \text{Top_Patient.SSN} = \text{People.SSN}$

$\text{People} \Pi \text{ People.FirstName, People.LastName, Patient_Totals.Total} (\text{Top_Person})$

New E:

$\text{Patients_Payments_CY} \leftarrow \text{Payments_CY} \bowtie \text{Payments_CY.SSN} = \text{Patients.SSN} \text{ Patients}$

$\text{Patient_Totals} \leftarrow \Pi \text{ Patients.SSN, SUM(PaymentAmount)} \rightarrow \text{Total} (\text{Patients_Payments_CY})$

$\text{Max_Total} \leftarrow \Pi \text{ MAX(Total)} \rightarrow \text{MaxValue} (\text{Patient_Totals})$

2) ERD/RA/Relational Schema

Updated Relational Algebra

A)

$\text{Patient_People} \leftarrow \text{People} \bowtie \text{People.SSN} = \text{Patients.SSN} \text{ Patients}$

$\text{Patient_Meds} \leftarrow \text{Patient_People} \bowtie \text{Patient_People.SSN} = \text{Medications.SSN} \text{ Medications}$

$\Pi \text{ FirstName, LastName, ActiveMeds} (\text{Patient_Meds})$

B)

$\text{Delta_Insurance} \leftarrow \sigma \text{ PolicyName} = \text{'Delta Dental'} (\text{Insurance})$

$\text{Delta_Patients} \leftarrow \text{Delta_Insurance} \bowtie \text{Delta_Insurance.PolicyID} = \text{Patients.PolicyID} \text{ Patients}$

$\text{Delta_People} \leftarrow \text{Delta_Patients} \bowtie \text{Delta_Patients.SSN} = \text{People.SSN} \text{ People}$

$\Pi \text{ FirstName, LastName, PolicyName} (\text{Delta_People})$

C)

$\text{Smilow_People} \leftarrow \sigma \text{ LastName} = \text{'Smilow'} (\text{People})$

$\text{Smilow_Staff} \leftarrow \text{Smilow_People} \bowtie \text{Smilow_People.SSN} = \text{MedicalStaff.SSN} \text{ MedicalStaff}$

$\text{Smilow_Performed} \leftarrow \text{Smilow_Staff} \bowtie \text{Smilow_Staff.SSN} =$

$\text{MedicalStaffPerformedOnTreatments.SSN} \text{ MedicalStaffPerformedOnTreatments}$

$\text{Smilow_Treatments} \leftarrow \text{Smilow_Performed} \bowtie \text{Smilow_Performed.TreatmentID} =$

$\text{Treatments.TreatmentID} \text{ Treatments}$

$\text{Smilow_Appointments} \leftarrow \text{Smilow_Treatments} \bowtie \text{Smilow_Treatments.TreatmentID} =$

$\text{Appointments.TreatmentID} \text{ Appointments}$

$\Pi \text{ Treatments.TreatmentName, Appointments.Date} (\text{Smilow_Appointments})$

D)

Past_Due_Billing $\leftarrow \sigma$ (Today - BillingRecord.Date > 30 $\wedge$ Balance > 10) (BillingRecord)

Past_Due_Payments $\leftarrow$ Past_Due_Billing $\bowtie$ Past_Due_Billing.BillingRecordID =
Payment.BillingRecordID Payment

Patient_Payments $\leftarrow$ Past_Due_Payments $\bowtie$ Past_Due_Payments.SSN = Patients.SSN
Patients

Pay_Pat_Cont $\leftarrow$ Patient_Payments $\bowtie$ Patient_Payments.SSN = People.SSN People
 Π People.FirstName, People.LastName, Payment.Balance, BillingRecord.Date (Pay_Pat_Cont)

E)

Patients_Payments_CY $\leftarrow$ Payments_CY $\bowtie$ Payments_CY.SSN = Patients.SSN Patients

Patient_Totals $\leftarrow \Pi$ Patients.SSN, SUM(PaymentAmount) $\rightarrow$ Total (Patients_Payments_CY)

Max_Total $\leftarrow \Pi$ MAX(Total) $\rightarrow$ MaxValue (Patient_Totals)

F)

Appointments_CY $\leftarrow \sigma$ (Today - Appointment.Date <= 365) Appointments

Treatments_CY $\leftarrow$ Appointments_CY $\bowtie$ Appointments_CY.TreatmentID =
Treatments.TreatmentID Treatments

Staff_Treatments $\leftarrow$ Treatments_CY $\bowtie$ Treatments_CY.TreatmentID =

MedicalStaffPerformedOnTreatments.TreatmentID MedicalStaffPerformedOnTreatments

Staff_Treat_Count $\leftarrow \gamma$ SSN, COUNT(TreatmentID) AS Total (Staff_Treatments)

Staff_Under_5 $\leftarrow \sigma$ (Total < 5) (Staff_Treat_Count)

Doctor_Under_5 $\leftarrow$ Staff_Under_5 $\bowtie$ Staff_Under_5.SSN = People.SSN People

Π People.FirstName, People.LastName, Total (Doctor_Under_5)

G)

Treatments_MedStaff $\leftarrow$ Treatments $\bowtie$ Treatments.TreatmentID =

MedicalStaffPerformedOnTreatments.TreatmentID MedicalStaffPerformedOnTreatments

All_Procedures $\leftarrow \gamma$ TreatmentID, COUNT(TreatmentID) (Treatments_MedStaff)

Max_Procedure $\leftarrow$ MAX(StandardizedCost) (Treatments)

Count_Max_Proc $\leftarrow$ All_Procedures $\bowtie$ All_Procedures.TreatmentID =

Max_Procedure.TreatmentID

Π Treatments.TreatmentID, Treatments.StandardizedCost, COUNT(TreatmentID)
(Count_Max_Proc)

H)

Payment_Counts $\leftarrow \gamma$ Type, COUNT(PaymentID) (Payment)

Payment_Totals $\leftarrow \gamma$ Type, SUM(PaymentAmount) (Payment)

Counts_Totals_Pmt $\leftarrow$ Payment_Counts $\bowtie$ Payment_Counts.Type = Payment_Totals.Type
Payment_Totals

Π Payment_Counts.Type, Payment_Counts.COUNT(PaymentID),

Payment_Totals.SUM(PaymentAmount) (Counts_Totals_Pmt)

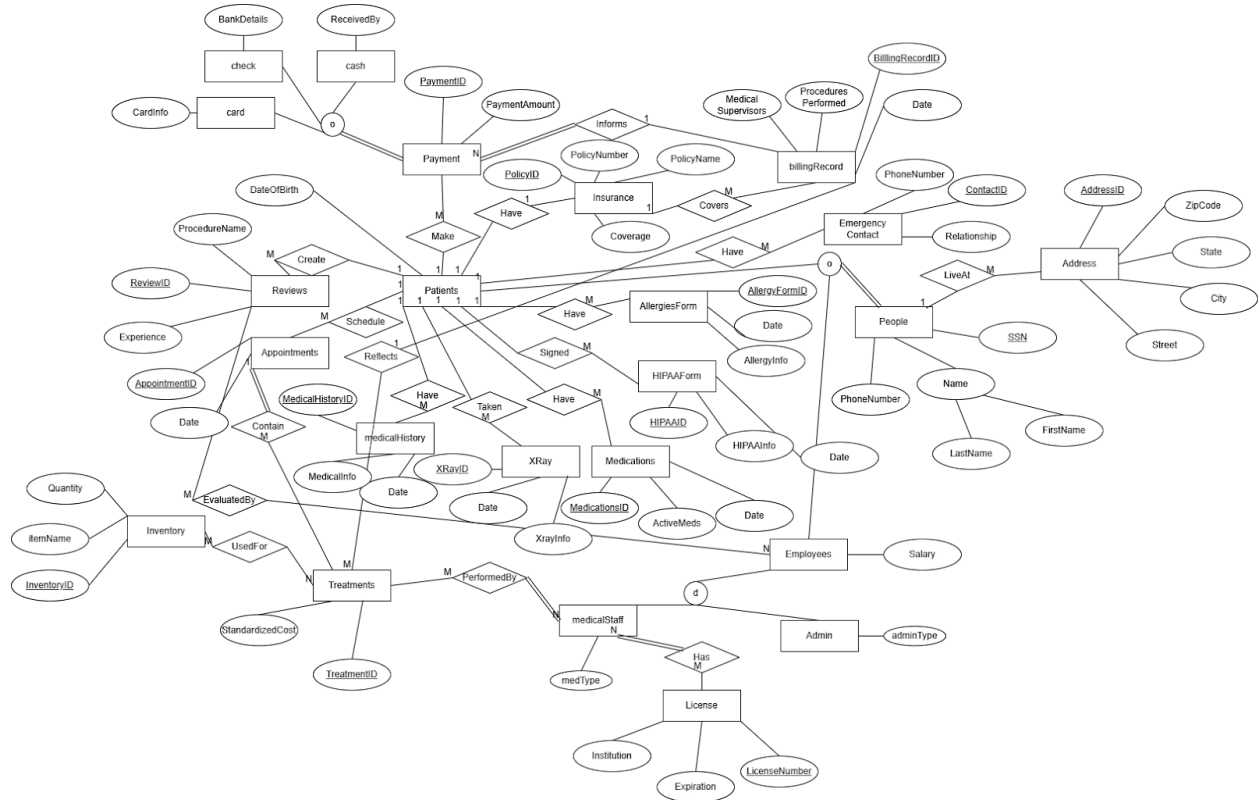
I)

Patient_Insurance $\leftarrow$ Patients $\bowtie$ Patients.PolicyID = Insurance.PolicyID Insurance

Plan_Counts $\leftarrow \gamma$ PolicyID, COUNT(SSN)(Patient_Insurance)

Π PolicyID, Insurance.PolicyName, Plan_Counts.COUNT(PolicyID) (Plan_Counts)

ERD Diagram



Relational Schema (All in BCNF)

Payment Table:

<u>Payment ID</u>	Payment Amount	Type	Received By	<u>SSN</u>	<u>Billing Record ID</u>
-------------------	----------------	------	-------------	------------	--------------------------

CardPayment Table:

<u>Payment ID</u> (FK to Payment, PK here as well)	CardInfo
--	----------

BankPayment Table:

<u>Payment ID</u> (FK to Payment, PK here as well)	BankInfo
--	----------

InsurancePayment Table:

<u>Payment ID</u> (FK to Payment, PK here as well)	PolicyNumber
--	--------------

Insurance

<u>PolicyID</u>	PolicyNumber	PolicyName
-----------------	--------------	------------

Primary: PolicyID

Reviews

<u>ReviewID</u>	Experience	ProcedureName	<u>SSN</u>
-----------------	------------	---------------	------------

Primary: ReviewID

Foreign: SSN -> People

Employee Table:

<u>SSN</u>	Employee Type	Salary
------------	---------------	--------

Primary: SSN

EmployeeEvaluatedByReviews

<u>ReviewID</u>	<u>SSN</u>
-----------------	------------

Primary: ReviewID, SSN

Foreign: ReviewID -> Reviews

SSN-> Employees

Admin

<u>SSN</u>	AdminType
------------	-----------

Primary: SSN

License

<u>LicenseNumber</u>	Expiration	Institution
----------------------	------------	-------------

Primary: LicenseNumber

MedicalStaff

<u>SSN</u>	MedType	Salary
------------	---------	--------

Primary: SSN

Billing Record

<u>BillingRecordID</u>	Date	Procedures Performed	MedicalSupervisors	<u>PolicyID</u>
------------------------	------	----------------------	--------------------	-----------------

Primary: BillingRecordID

Foreign: PolicyID -> Insurance

MedicalStaffHasLicense

<u>SSN</u>	<u>LicenseNumber</u>
------------	----------------------

Primary: SSN, LicenseNumber

Foreign: SSN-> Employees

LicenseNumber-> License

MedicalStaffPerformedOnTreatments

<u>SSN</u>	<u>TreatmentID</u>
------------	--------------------

Primary: SSN, TreatmentID

Foreign: SSN-> Employees

TreatmentID -> Treatment

Inventory

<u>InventoryID</u>	Quantity	ItemName
--------------------	----------	----------

Primary: InventoryID

Treatments

<u>TreatmentID</u>	StandardizedCost	EstimatedTime
--------------------	------------------	---------------

Primary: TreatmentID

InventoryUsedForTreatments

<u>ItemName</u>	<u>TreatmentID</u>	QuantityUsed
-----------------	--------------------	--------------

Primary: ItemName, TreatmentID,

Foreign: ItemName-> Inventory

TreatmentID-> Treatment

Appointments

<u>AppointmentID</u>	Date	<u>SSN</u>	<u>TreatmentID</u>
----------------------	------	------------	--------------------

Primary: AppointmentID

Foreign: SSN-> People

Patients

<u>SSN</u>	DateOfBirth	<u>PolicyID</u>
------------	-------------	-----------------

Primary: SSN

Foreign: PolicyID-> Insurance

People

<u>SSN</u>	FirstName	LastName	PhoneNumber
------------	-----------	----------	-------------

Primary: SSN

EmergencyContact

<u>EmergencyContactID</u>	FirstName	LastName	PhoneNumber	Relationship
---------------------------	-----------	----------	-------------	--------------

Primary: EmergencyContactID

Address

<u>AddressID</u>	Street	City	State	ZipCode
------------------	--------	------	-------	---------

Primary: AddressID

PeopleLiveAtAddresses

<u>SSN</u>	<u>AddressID</u>
------------	------------------

Primary: SSN, AddressID

Foreign: SSN-> People

AddressID -> Address

MedicalHistory

<u>MedicalHistoryID</u>	MedicalInfo	Date	<u>SSN</u>
-------------------------	-------------	------	------------

Primary: MedicalHistoryID

Foreign: SSN-> People

Xray

<u>XRayID</u>	XRayInfo	Date	<u>SSN</u>
---------------	----------	------	------------

Primary: XRayID

Foreign: SSN-> People

Medications

<u>MedicationsID</u>	ActiveMeds	Date	<u>SSN</u>
----------------------	------------	------	------------

Primary: MedicationsID

Foreign: SSN-> People

AllergyForm

<u>AllergyFormID</u>	AllergyInfo	Date	<u>SSN</u>
----------------------	-------------	------	------------

Primary: AllergyFormID

Foreign: SSN-> People

HIPAAForm

<u>HIPAAFormID</u>	HIPAAInfo	Date	<u>SSN</u>
--------------------	-----------	------	------------

Primary: HIPAAFormID

Foreign: SSN-> People

3. Previous .txt files from Project 3

We used SQLiteonline.com

[BEGIN CreateQueries.txt]

```
-- Drop tables if they exist (for re-creation purposes)
DROP TABLE IF EXISTS Payment;
DROP TABLE IF EXISTS CardPayment;
DROP TABLE IF EXISTS BankPayment;
DROP TABLE IF EXISTS InsurancePayment;
DROP TABLE IF EXISTS Insurance;
DROP TABLE IF EXISTS Reviews;
DROP TABLE IF EXISTS Employees;
DROP TABLE IF EXISTS EmployeeEvaluatedByReviews;
DROP TABLE IF EXISTS Admin;
DROP TABLE IF EXISTS License;
DROP TABLE IF EXISTS MedicalStaff;
DROP TABLE IF EXISTS BillingRecord;
DROP TABLE IF EXISTS MedicalStaffHasLicense;
DROP TABLE IF EXISTS MedicalStaffPerformedOnTreatments;
DROP TABLE IF EXISTS Inventory;
DROP TABLE IF EXISTS Treatments;
DROP TABLE IF EXISTS InventoryUsedForTreatments;
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS People;
DROP TABLE IF EXISTS EmergencyContact;
DROP TABLE IF EXISTS Address;
DROP TABLE IF EXISTS PeopleLiveAtAddresses;
DROP TABLE IF EXISTS MedicalHistory;
DROP TABLE IF EXISTS XRay;
DROP TABLE IF EXISTS Medications;
DROP TABLE IF EXISTS AllergyForm;
```

```
DROP TABLE IF EXISTS HIPAAForm;
```

```
DROP INDEX IF EXISTS idx_payment_type;
```

```
DROP INDEX IF EXISTS idx_payment_recievedby;
```

```
CREATE TABLE IF NOT EXISTS People (  
    SSN CHAR(9) PRIMARY KEY,  
    FirstName VARCHAR(100) NOT NULL,  
    LastName VARCHAR(100) NOT NULL,  
    PhoneNumber VARCHAR(15)  
);
```

```
CREATE TABLE IF NOT EXISTS Address (  
    AddressID INT PRIMARY KEY,  
    Street TEXT NOT NULL,  
    City TEXT NOT NULL,  
    State TEXT NOT NULL,  
    ZipCode CHAR(5)  
);
```

```
CREATE TABLE IF NOT EXISTS PeopleLiveAtAddresses (  
    SSN CHAR(9),  
    AddressID INT,  
    PRIMARY KEY (SSN, AddressID),  
    FOREIGN KEY (SSN) REFERENCES People(SSN),  
    FOREIGN KEY (AddressID) REFERENCES Address(AddressID)  
);
```

```
CREATE TABLE IF NOT EXISTS Insurance (  
    PolicyID INT PRIMARY KEY,  
    PolicyNumber VARCHAR(50) NOT NULL UNIQUE,  
    PolicyName TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS Patients (  
    SSN CHAR(9) PRIMARY KEY,  
    PolicyID INT,  
    DateOfBirth DATE NOT NULL,  
    FOREIGN KEY (SSN) REFERENCES People(SSN),  
    FOREIGN KEY (PolicyID) REFERENCES Insurance(PolicyID)  
);
```

```
CREATE TABLE IF NOT EXISTS MedicalStaff (  
    SSN CHAR(9) PRIMARY KEY,
```



```

        MedType VARCHAR(50) NOT NULL,
        Salary FLOAT(10, 2),
        FOREIGN KEY (SSN) REFERENCES People(SSN)
    );

CREATE TABLE IF NOT EXISTS License (
    LicenseNumber INT PRIMARY KEY,
    Expiration DATE NOT NULL,
    Institution TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS MedicalStaffHasLicense (
    SSN CHAR(9),
    LicenseNumber INT,
    PRIMARY KEY (SSN, LicenseNumber),
    FOREIGN KEY (LicenseNumber) REFERENCES License(LicenseNumber),
    FOREIGN KEY (SSN) REFERENCES People(SSN)
);

CREATE TABLE IF NOT EXISTS Treatments (
    TreatmentID INT PRIMARY KEY,
    StandardizedCost FLOAT(10, 2) NOT NULL,
    EstimatedTime INT NOT NULL
);

CREATE TABLE IF NOT EXISTS BillingRecord (
    BillingRecordID INT PRIMARY KEY,
    Date DATE NOT NULL,
    ProceduresPerformed INT NOT NULL,
    MedicalSupervisors TEXT NOT NULL,
    PolicyID INT,
    FOREIGN KEY (PolicyID) REFERENCES Insurance(PolicyID)
);

CREATE TABLE IF NOT EXISTS Payment (
    PaymentID INT PRIMARY KEY,
    PaymentAmount DECIMAL(10, 2) NOT NULL,
    Type VARCHAR(50) CHECK (Type IN ('Cash', 'Check', 'Card',
'Insurance')),
    ReceivedBy VARCHAR(100) NOT NULL,
    SSN CHAR(9),
    BillingRecordID INT,
    FOREIGN KEY (SSN) REFERENCES People(SSN),
    FOREIGN KEY (BillingRecordID) REFERENCES
BillingRecord(BillingRecordID)

```

```
);
```

```
CREATE TABLE IF NOT EXISTS CardPayment (  
    PaymentID INT PRIMARY KEY,  
    CardInfo VARCHAR(50) NOT NULL,  
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)  
);
```

```
CREATE TABLE IF NOT EXISTS BankPayment (  
    PaymentID INT PRIMARY KEY,  
    BankInfo VARCHAR(50) NOT NULL,  
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)  
);
```

```
CREATE TABLE IF NOT EXISTS InsurancePayment (  
    PaymentID INT PRIMARY KEY,  
    PolicyNumber VARCHAR(50) NOT NULL,  
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)  
);
```

```
CREATE TABLE IF NOT EXISTS Reviews (  
    ReviewID INT PRIMARY KEY,  
    Experience TEXT NOT NULL,  
    ProcedureName VARCHAR(100),  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS Employees (  
    SSN CHAR(9) PRIMARY KEY,  
    EmployeeType VARCHAR(50) NOT NULL,  
    Salary FLOAT(10, 2),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS EmployeeEvaluatedByReviews (  
    SSN CHAR(9) PRIMARY KEY,  
    ReviewID INT NOT NULL,  
    FOREIGN KEY (ReviewID) REFERENCES Reviews(ReviewID),  
    FOREIGN KEY (SSN) REFERENCES Employees(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS Admin (  
    SSN CHAR(9) PRIMARY KEY,  
    AdminType VARCHAR(50) NOT NULL,
```

```
        Salary FLOAT(10, 2),  
        FOREIGN KEY (SSN) REFERENCES People(SSN)  
    );
```

```
CREATE TABLE IF NOT EXISTS MedicalHistory (  
    MedicalHistoryID INT PRIMARY KEY,  
    MedicalInfo TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS XRay (  
    XRayID INT PRIMARY KEY,  
    XRayInfo TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS Medications (  
    MedicationsID INT PRIMARY KEY,  
    ActiveMeds TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS AllergyForm (  
    AllergyFormID INT PRIMARY KEY,  
    AllergyInfo TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS HIPAAForm (  
    HIPAAFormID INT PRIMARY KEY,  
    HIPAAInfo TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```
CREATE TABLE IF NOT EXISTS Appointments (  
    AppointmentID INT PRIMARY KEY,  
    AppointmentInfo TEXT NOT NULL,  
    Date DATE NOT NULL,  
    SSN CHAR(9),  
    FOREIGN KEY (SSN) REFERENCES People(SSN)  
);
```

```

AppointmentID INT PRIMARY KEY,
Date DATE NOT NULL,
SSN CHAR(9),
TreatmentID INT,
FOREIGN KEY (SSN) REFERENCES People(SSN),
FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID)
);

```

```

CREATE TABLE IF NOT EXISTS Inventory (
    ItemName VARCHAR(256) PRIMARY KEY,
    Quantity INT NOT NULL
);

```

```

CREATE TABLE IF NOT EXISTS InventoryUsedForTreatments (
    ItemName VARCHAR(256),
    TreatmentID INT,
    QuantityUsed INT NOT NULL,
    PRIMARY KEY (ItemName, TreatmentID),
    FOREIGN KEY (ItemName) REFERENCES Inventory(ItemName),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID)
);

```

```

CREATE TABLE IF NOT EXISTS MedicalStaffPerformedOnTreatments (
    SSN CHAR(9),
    TreatmentID INT,
    PRIMARY KEY (SSN, TreatmentID),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID),
    FOREIGN KEY (SSN) REFERENCES People(SSN)
);

```

```

CREATE INDEX IF NOT EXISTS idx_payment_type ON Payment(Type);
CREATE INDEX IF NOT EXISTS idx_payment_receivedby ON
Payment(ReceivedBy);

```

[END CreateQueries.txt]

[BEGIN InsertQueries.txt]

```

INSERT INTO People (SSN, FirstName, LastName, PhoneNumber) VALUES
('123456789', 'John', 'Doe', '555-0123'),
('234567890', 'Jane', 'Smith', '555-0456'),
('345678901', 'Emily', 'Jones', '555-0789'),
('456789012', 'Michael', 'Brown', '555-1456'),
('567890123', 'Sarah', 'Davis', '555-1280'),
('678901234', 'Christopher', 'Taylor', '555-2356'),
('789012345', 'Jessica', 'Wilson', '555-3456'),
('890123456', 'David', 'Lee', '555-6789'),

```

```
('901234567', 'Jennifer', 'Martinez', '555-7890'),  
('123456780', 'William', 'Garcia', '555-0129'),  
('999999999', 'John', 'Smilow', '555-9999');
```

```
INSERT INTO Address (AddressID, Street, City, State, ZipCode) VALUES  
(1, '123 Elm St', 'Springfield', 'IL', '62701'),  
(2, '456 Maple Rd', 'Springfield', 'IL', '62702'),  
(3, '789 Oak Ave', 'Springfield', 'IL', '62703'),  
(4, '321 Pine St', 'Springfield', 'IL', '62704'),  
(5, '654 Cedar St', 'Springfield', 'IL', '62705');
```

```
INSERT INTO PeopleLiveAtAddresses (SSN, AddressID) VALUES  
('123456789', 1),  
('234567890', 2),  
('345678901', 3),  
('456789012', 4),  
('567890123', 5),  
('678901234', 1),  
('789012345', 2),  
('890123456', 3),  
('901234567', 4),  
('123456780', 5);
```

```
INSERT INTO Insurance (PolicyID, PolicyNumber, PolicyName) VALUES  
(1, 'POL-1001', 'Basic Health Plan'),  
(2, 'POL-1002', 'Basic Health Plan'),  
(3, 'POL-1003', 'Dental Care Plan'),  
(4, 'POL-1004', 'Vision Care Plan'),  
(5, 'POL-1005', 'Family Health Plan'),  
(6, 'POL-1006', 'Senior Citizen Plan'),  
(7, 'POL-1007', 'OSU health plan');
```

```
INSERT INTO Patients (SSN, PolicyID, DateOfBirth) VALUES  
('123456789', 1, '1980-01-01'),  
('234567890', 2, '1990-02-02'),  
('345678901', 3, '1985-03-03'),  
('456789012', 4, '1972-04-04'),  
('567890123', 5, '2000-05-05');
```

```
INSERT INTO MedicalStaff (SSN, MedType, Salary) VALUES  
('678901234', 'Nurse', 60000.00),  
('789012345', 'Doctor', 120000.00),  
('890123456', 'Technician', 50000.00),  
('901234567', 'Therapist', 70000.00),
```

```
('123456780', 'Surgeon', 150000.00),  
( '999999999', 'Doctor', 120000.00);
```

```
INSERT INTO License (LicenseNumber, Expiration, Institution) VALUES  
(1, '2024-12-31', 'Medical Institution A'),  
(2, '2025-06-30', 'Medical Institution B'),  
(3, '2023-09-30', 'Medical Institution C'),  
(4, '2024-11-15', 'Medical Institution D'),  
(5, '2025-03-01', 'Medical Institution E');
```

```
INSERT INTO MedicalStaffHasLicense (SSN, LicenseNumber) VALUES  
( '678901234', 1),  
( '789012345', 2),  
( '890123456', 3),  
( '901234567', 4),  
( '123456780', 5);
```

```
INSERT INTO Treatments (TreatmentID, StandardizedCost, EstimatedTime)  
VALUES  
(1, 200.00, 30), --Kidney Surgery  
(2, 150.00, 45), -- Xray  
(3, 300.00, 60), -- Leg surgery  
(4, 75.00, 15), --Check up  
(5, 500.00, 90); --Heart surgery
```

```
INSERT INTO BillingRecord (BillingRecordID, Date, ProceduresPerformed,  
MedicalSupervisors, PolicyID) VALUES  
(1, '2023-10-01', 1, 'Dr. Roberts', 1), -- Matches TreatmentID 1, cost  
200.00  
(2, '2023-10-02', 2, 'Dr. Smith', 2), -- Matches TreatmentID 2, cost  
150.00  
(3, '2023-10-03', 3, 'Dr. Johnson', 3), -- Matches TreatmentID 3, cost  
300.00  
(4, '2023-10-05', 4, 'Dr. Brown', 4), -- Matches TreatmentID 4, cost  
75.00  
(5, '2023-10-06', 5, 'Dr. Lee', 5), -- Matches TreatmentID 5, cost  
500.00  
(6, '2023-10-07', 1, 'Dr. Roberts', 1), -- Matches TreatmentID 1, cost  
200.00  
(7, '2023-10-08', 2, 'Dr. Smith', 2), -- Matches TreatmentID 2, cost  
150.00  
(8, '2023-10-09', 3, 'Dr. Johnson', 3), -- Matches TreatmentID 3, cost  
300.00  
(9, '2023-10-10', 4, 'Dr. Brown', 4), -- Matches TreatmentID 4, cost  
75.00
```

```
(10, '2023-10-11', 5, 'Dr. Lee', 5);    -- Matches TreatmentID 5, cost
500.00
```

```
INSERT INTO Payment (PaymentID, PaymentAmount, Type, ReceivedBy, SSN,
BillingRecordID) VALUES
```

```
(1, 200.00, 'Cash', 'Receptionist A', '123456789', 1), -- Matches
TreatmentID 1, cost 200.00
(2, 150.00, 'Cash', 'Receptionist B', '234567890', 2), -- Matches
TreatmentID 2, cost 150.00
(3, 300.00, 'Card', 'Receptionist B', '345678901', 3), -- Matches
TreatmentID 3, cost 300.00
(4, 75.00, 'Check', 'Receptionist D', '456789012', 4), -- Matches
TreatmentID 4, cost 75.00
(5, 500.00, 'Card', 'Receptionist E', '567890123', 5), -- Matches
TreatmentID 5, cost 500.00
(6, 200.00, 'Cash', 'Receptionist A', '678901234', 6), -- Matches
TreatmentID 1, cost 200.00
(7, 150.00, 'Card', 'Receptionist B', '789012345', 7), -- Matches
TreatmentID 2, cost 150.00
(8, 300.00, 'Card', 'Receptionist C', '890123456', 8), -- Matches
TreatmentID 3, cost 300.00
(9, 75.00, 'Check', 'Receptionist D', '901234567', 9), -- Matches
TreatmentID 4, cost 75.00
(10, 500.00, 'Cash', 'Receptionist E', '123456789', 10); -- Matches
TreatmentID 5, cost 500.00
```

```
INSERT INTO CardPayment (PaymentID, CardInfo) VALUES
```

```
(2, 'Visa ending in 1234'),
(5, 'Mastercard ending in 5678'),
(8, 'Amex ending in 9012');
```

```
INSERT INTO BankPayment (PaymentID, BankInfo) VALUES
```

```
(9, 'Bank of America, Account #123456789');
```

```
INSERT INTO InsurancePayment (PaymentID, PolicyNumber) VALUES
```

```
(3, 'INS-23456'),
(7, 'INS-34567');
```

```
INSERT INTO Reviews (ReviewID, Experience, ProcedureName, SSN) VALUES
```

```
(1, 'Excellent service and care.', 'X-Ray', '123456789'),
(2, 'Very professional and punctual.', 'MRI', '234567890'),
(3, 'The staff was friendly.', 'Physical Therapy', '345678901'),
(4, 'Highly skilled doctors.', 'Surgery', '456789012'),
```

```
(5, 'Clean facility and good atmosphere.', 'Routine Checkup',  
'567890123');
```

```
INSERT INTO Employees (SSN, EmployeeType, Salary) VALUES  
( '678901234', 'Nurse', 60000.00),  
( '789012345', 'Surgical Assistant', 50000.00),  
( '890123456', 'Admin Staff', 40000.00),  
( '901234567', 'Front Desk', 35000.00),  
( '123456780', 'Medical Assistant', 30000.00);
```

```
INSERT INTO EmployeeEvaluatedByReviews (SSN, ReviewID) VALUES  
( '678901234', 1),  
( '789012345', 2),  
( '890123456', 3),  
( '901234567', 4),  
( '123456780', 5);
```

```
INSERT INTO Admin (SSN, AdminType, Salary) VALUES  
( '123456789', 'Head Admin', 80000.00),  
( '234567890', 'Assistant Admin', 45000.00);
```

```
INSERT INTO MedicalHistory (MedicalHistoryID, MedicalInfo, Date, SSN)  
VALUES  
(1, 'Allergic to Penicillin', '2022-01-15', '123456789'),  
(2, 'Has Asthma', '2022-02-20', '234567890'),  
(3, 'Diabetic', '2022-03-18', '345678901'),  
(4, 'High Blood Pressure', '2022-04-10', '456789012'),  
(5, 'No known allergies', '2022-05-12', '567890123');
```

```
INSERT INTO XRay (XRayID, XRayInfo, Date, SSN) VALUES  
(1, 'Fracture of left arm.', '2023-10-01', '123456789'),  
(2, 'Chest X-Ray.', '2023-10-02', '234567890'),  
(3, 'Back X-Ray show alignment issue.', '2023-10-03', '345678901'),  
(4, 'Knee X-Ray shows minor injury.', '2023-10-04', '456789012'),  
(5, 'Dental X-Ray shows no issues.', '2023-10-05', '567890123');
```

```
INSERT INTO Medications (MedicationsID, ActiveMeds, Date, SSN) VALUES  
(1, 'Lisinopril', '2023-01-01', '123456789'),  
(2, 'Metformin', '2023-01-05', '234567890'),  
(3, 'Albuterol', '2023-01-10', '345678901'),  
(4, 'Insulin', '2023-01-15', '456789012'),  
(5, 'Aspirin', '2023-01-20', '567890123');
```

```
INSERT INTO AllergyForm (AllergyFormID, AllergyInfo, Date, SSN) VALUES  
(1, 'No allergies', '2023-01-01', '123456789'),
```



```
(2, 'Shellfish allergy', '2023-01-05', '234567890'),  
(3, 'Nut allergy', '2023-01-10', '345678901'),  
(4, 'Pollen allergy', '2023-01-15', '456789012'),  
(5, 'Dust allergy', '2023-01-20', '567890123');
```

```
INSERT INTO HIPAAForm (HIPAAFormID, HIPAAInfo, Date, SSN) VALUES  
(1, 'Patient consent on file', '2023-01-01', '123456789'),  
(2, 'All documents signed', '2023-01-05', '234567890'),  
(3, 'Patient understood rights', '2023-01-10', '345678901'),  
(4, 'Confidentiality agreed', '2023-01-15', '456789012'),  
(5, 'Informed about records handling', '2023-01-20', '567890123');
```

```
INSERT INTO Appointments (AppointmentID, Date, SSN, TreatmentID)  
VALUES  
(1, '2023-10-15', '123456789', 1),  
(2, '2023-10-16', '234567890', 2),  
(3, '2023-10-17', '345678901', 3),  
(4, '2023-10-18', '456789012', 4),  
(5, '2023-10-19', '567890123', 5);
```

```
INSERT INTO Inventory (ItemName, Quantity) VALUES  
( 'Bandage', 100),  
( 'Gauze', 200),  
( 'Adhesive Tape', 150),  
( 'Scalpel', 75),  
( 'Syringe', 500);
```

```
INSERT INTO InventoryUsedForTreatments (ItemName, TreatmentID,  
QuantityUsed) VALUES  
( 'Bandage', 1, 5),  
( 'Gauze', 2, 3),  
( 'Adhesive Tape', 3, 10),  
( 'Scalpel', 4, 1),  
( 'Syringe', 5, 2);
```

```
INSERT INTO MedicalStaffPerformedOnTreatments (SSN, TreatmentID)  
VALUES  
( '678901234', 1),  
( '789012345', 2),  
( '890123456', 3),  
( '999999999', 4),  
( '999999999', 5);
```

[END InsertQueries.txt]

[BEGIN SimpleQueries.txt]

-- Query 1: Retrieve Patients with Medications

```
SELECT
    Patients.SSN,
    People.FirstName,
    People.LastName,
    Medications.MedicationsID,
    Medications.ActiveMeds,
    Medications.Date
FROM
    Patients
JOIN
    People ON Patients.SSN = People.SSN
JOIN
    Medications ON Patients.SSN = Medications.SSN
ORDER BY
    People.LastName ASC, Medications.ActiveMeds ASC;
```

-- Query 2: Retrieve Patients with Insurance

```
SELECT
    Patients.SSN,
    Patients.DateOfBirth,
    People.FirstName,
    People.LastName,
    People.PhoneNumber,
    Patients.PolicyID
FROM
    Patients
JOIN
    Insurance ON Patients.PolicyID = Insurance.PolicyID
JOIN
    People ON Patients.SSN = People.SSN
WHERE
    Insurance.PolicyNumber = 'POL-1001';
```

-- Query 3:

```
SELECT
    Treatments.TreatmentID,
    Treatments.StandardizedCost,
    Treatments.EstimatedTime,
    Appointments.Date
FROM
    MedicalStaffPerformedOnTreatments
JOIN
```

```

        MedicalStaff ON MedicalStaffPerformedOnTreatments.SSN =
MedicalStaff.SSN
    JOIN
        Treatments ON MedicalStaffPerformedOnTreatments.TreatmentID =
Treatments.TreatmentID
    JOIN
        Appointments ON Treatments.TreatmentID = Appointments.TreatmentID
    JOIN
        People ON MedicalStaff.SSN = People.SSN
WHERE
    MedicalStaff.MedType = 'Doctor' AND People.LastName = 'Smilow';

```

-- Query 4: Retrieve Billing Records for Patients

```

SELECT
    br.BillingRecordID,
    br.Date AS InvoiceDate,
    br.ProceduresPerformed,
    br.MedicalSupervisors,
    p.SSN,
    p.FirstName,
    p.LastName,
    p.PhoneNumber
FROM
    BillingRecord br
JOIN
    Patients pt ON br.PolicyID = pt.PolicyID
JOIN
    People p ON pt.SSN = p.SSN
WHERE
    br.Date < CURRENT_DATE;

```

-- Query 5: Calculate Revenue for Patients

```

SELECT
    People.FirstName,
    People.LastName,
    Patients.SSN,
    SUM(Payment.PaymentAmount) AS Revenue
FROM
    Payment
JOIN
    Patients ON Patients.SSN = Payment.SSN
JOIN
    People ON Patients.SSN = People.SSN
GROUP BY

```

```
Patients.SSN
ORDER BY
    Revenue DESC
LIMIT 5;
```

```
-- Query 6: Count Procedures Managed by Medical Supervisors
WITH Treatments_All AS (
    SELECT a.*, t.StandardizedCost, t.EstimatedTime
    FROM Appointments a
    JOIN Treatments t ON a.TreatmentID = t.TreatmentID
),
Staff_Treatments AS (
    SELECT ta.*, mst.SSN
    FROM Treatments_All ta
    JOIN MedicalStaffPerformedOnTreatments mst ON ta.TreatmentID =
mst.TreatmentID
),
Staff_Treat_Count AS (
    SELECT
        SSN,
        COUNT(TreatmentID) AS Total
    FROM Staff_Treatments
    GROUP BY SSN
),
Staff_Under_5 AS (
    SELECT *
    FROM Staff_Treat_Count
    WHERE Total < 5
),
Doctor_Under_5 AS (
    SELECT su5.Total, p.FirstName, p.LastName
    FROM Staff_Under_5 su5
    JOIN People p ON su5.SSN = p.SSN
)
SELECT
    FirstName,
    LastName,
    Total
FROM Doctor_Under_5;
```

```
-- Query 7: Get the Most Frequent Procedure and Its Total Payment
SELECT
    br.ProceduresPerformed,
```

```

        COUNT(br.ProceduresPerformed) AS ProcedureCount,
        t.StandardizedCost AS ProcedurePrice,
        SUM(p.PaymentAmount) AS TotalPayment
FROM
    BillingRecord br
JOIN
    Payment p ON br.BillingRecordID = p.BillingRecordID
JOIN
    Treatments t ON CAST(br.ProceduresPerformed AS INT) = t.TreatmentID
GROUP BY
    br.ProceduresPerformed,
    t.StandardizedCost
ORDER BY
    TotalPayment DESC;

```

-- Query 8: Summary of Payment Types

```

SELECT
    Type,
    COUNT(*) AS PaymentCount,
    SUM(PaymentAmount) AS TotalAmount
FROM
    Payment
GROUP BY
    Type;

```

-- Query 9: Insurance Plan Count

```

SELECT
    Insurance.PolicyName,
    COUNT(Patients.SSN) AS Plan_Count
FROM
    Patients
JOIN
    Insurance ON Patients.PolicyID = Insurance.PolicyID
GROUP BY
    Insurance.PolicyName
ORDER BY
    Plan_Count DESC
LIMIT 1;

```

[END SimpleQueries.txt]

[BEGIN ExtraQueries.txt]

-- Query 1: List the Most Common Allergies of Patients

-- Description: Create a list of the most common allergies reported by patients, sorted by frequency.

```
SELECT
    AllergyInfo,
    COUNT(AllergyInfo) AS Frequency
FROM
    People
JOIN
    AllergyForm ON People.SSN = AllergyForm.SSN
GROUP BY
    AllergyInfo
ORDER BY
    Frequency DESC;
```

-- Query 2: List all patients along with their most recent appointment date, including those who have never made an appointment.

-- Description: Retrieve a list of all patients with their most recent appointment date, or NULL if they haven't scheduled any.

```
SELECT
    patient.SSN,
    people.FirstName,
    people.LastName,
    MAX(appointment.Date) AS MostRecentAppointment
FROM
    Patients patient
LEFT JOIN
    Appointments appointment ON patient.SSN = appointment.SSN
JOIN
    People people ON patient.SSN = people.SSN
GROUP BY
    patient.SSN, people.FirstName, people.LastName
ORDER BY
    people.LastName ASC, people.FirstName ASC;
```

-- Query 3: Identify medical staff who haven't performed any treatments in the past year, along with their job titles.

-- Description: Find all medical staff and their job titles who have not performed treatments in the past year.

```
SELECT
    staff.SSN,
    people.FirstName,
    people.LastName,
    staff.MedType AS JobTitle
FROM
    MedicalStaff staff
JOIN
    People people ON staff.SSN = people.SSN
LEFT JOIN
    MedicalStaffPerformedOnTreatments mspt ON staff.SSN = mspt.SSN
LEFT JOIN
    Treatments treatment ON mspt.TreatmentID = treatment.TreatmentID
WHERE
    treatment.TreatmentID IS NULL
    OR treatment.EstimatedTime < DATE('now', '-1 year')
GROUP BY
    staff.SSN, people.FirstName, people.LastName, staff.MedType
ORDER BY
    people.LastName ASC, people.FirstName ASC;
```

--BTree Query to see who handled cash payments, how many & how much

```
SELECT
    ReceivedBy,
    COUNT(*) AS CashPayments,
    SUM(PaymentAmount) AS TotalCashHandled
FROM
    Payment
WHERE
    Type = 'Cash'
GROUP BY
    ReceivedBy
ORDER BY
    TotalCashHandled DESC;
```

--Hash Index Query to search people based on phone number

```
SELECT *
FROM People
```

```
WHERE PhoneNumber = '555-0123';
```

```
[END ExtraQueries.txt]
```

4. InsertDeleteQueries.txt:

```
--do 3 inserts
```

```
Insert INTO PEOPLE (SSN, Firstname, Lastname, Phonenumber)  
VALUES('987987987', 'Alex', 'Johnson', '555-9876');
```

```
INSERT INTO Insurance(policyid, policynumber, policyname)  
VALUES (8, 'POL-1008', 'Mario Universal Life Plan');
```

```
INSERT INTO Patients (SSN, PolicyID, DateOfBirth)  
VALUES ('123123123', 8, '1990-05-05');
```

```
-- 3 DELETE STATEMENTS (in correct order):
```

```
DELETE FROM Patients WHERE SSN = '123123123';
```

```
DELETE FROM Insurance WHERE PolicyID = 8;
```

```
DELETE FROM People WHERE SSN = '987987987';
```

Evidence:

The screenshot displays a SQLite IDE interface. The main editor window contains SQL code for inserting data into three tables: PEOPLE, Insurance, and Patients. The code includes comments and SQL statements for inserting specific records. Below the code, a table view shows the data inserted into the PEOPLE table, with columns SSN, FirstName, LastName, and PhoneNumber. The table contains one row with the values 987987987, Alex, Johnson, and 555-9876. On the right side, a History panel shows a list of executed SQL queries, including the insert statements and subsequent select queries to verify the data.

```
442  
443 --do 3 inserts  
444 INSERT INTO PEOPLE (SSN, Firstname, Lastname, Phonenumber)  
445 VALUES('987987987', 'Alex', 'Johnson', '555-9876');  
446  
447 INSERT INTO Insurance(policyid, policynumber, policyname)  
448 VALUES (8, 'POL-1008', 'Mario Universal Life Plan');  
449  
450 INSERT INTO Patients (SSN, PolicyID, DateOfBirth)  
451 VALUES ('123123123', 8, '1990-05-05');  
452  
453  
454 -- See if data got properly inserted  
455 SELECT * FROM People WHERE SSN = '987987987';  
456 SELECT * FROM Insurance WHERE policyid = 8;  
457 SELECT * FROM Patients WHERE SSN = '123123123'  
458  
459  
460  
461  
462  
463  
464  
465  
...
```

SSN	FirstName	LastName	PhoneNumber
987987987	Alex	Johnson	555-9876

History

Syntax | History

SQLite

```
-- Drop tables if they exist (for re-crv  
DROP TABLE IF EXISTS Payment;  
DROP TABLE IF  
...
```

18:13:29

SQLite

```
-- Drop tables if they exist (for re-crv  
DROP TABLE IF EXISTS Payment;  
DROP TABLE IF  
...
```

18:11:19

SQLite

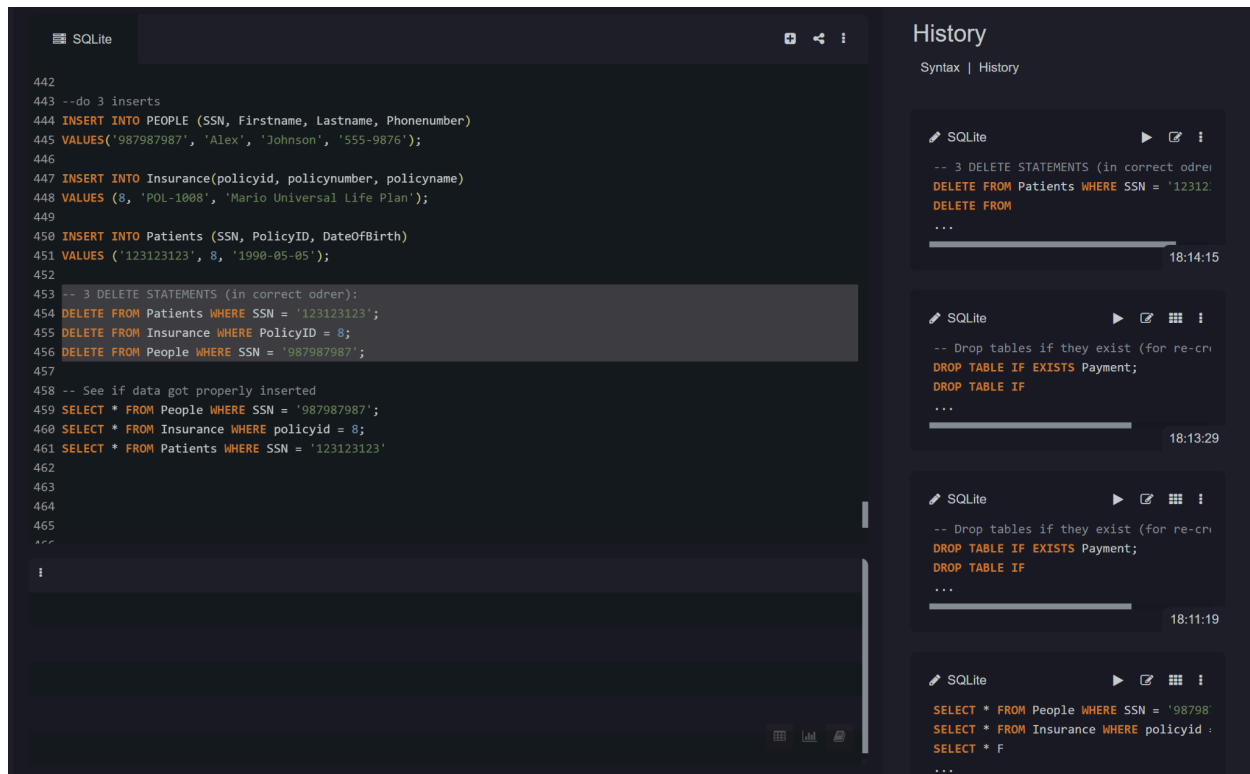
```
SELECT * FROM People WHERE SSN = '98798  
SELECT * FROM Insurance WHERE policyid :  
SELECT * F  
...
```

18:10:54

SQLite

```
SELECT * FROM People WHERE SSN = '98798'  
SELECT * FROM Insurance WHERE policyid :  
SELECT * F
```

Shows that inserts were successful

The screenshot shows a SQLite database interface. The main window displays a series of SQL queries, including INSERT statements for 'PEOPLE', 'Insurance', and 'Patients' tables, followed by DELETE statements to remove data from these tables. The queries are numbered 442 through 466. A 'History' panel on the right side of the interface shows a list of previously executed queries, including the same DELETE statements and a SELECT query. The interface has a dark theme and includes standard database management icons at the bottom.

Delete statements and look for the same data again, but nothing comes up as planned. DB still in valid state

5. Indexes

My first suggestion was to provide an index of all the receptionists who handled cash payments. I did this because sometimes in small businesses the cash register might be short on the exact money it should have, thus it is important to see who handled the cash payments to review what went wrong or if there was a miscalculation when handing back change. This just makes it easier to manage and track cash within the business. This index would use B - tree indexing as it would efficiently support exact matching (need to filter payments for only “Cash” by using Type = ‘Cash’) and querying through the range of names through the “ReceivedBy” column.

So for the SQL code...(Added in the end of ExtraQueries.txt)

Before the creation of the table we need to add:

```
DROP INDEX IF EXISTS idx_payment_type;  
DROP INDEX IF EXISTS idx_payment_recievedby;
```

After the creation of the tables we need to add:

```
CREATE INDEX IF NOT EXISTS idx_payment_type ON Payment(Type);  
CREATE INDEX IF NOT EXISTS idx_payment_receivedby ON Payment(ReceivedBy);
```

To query the this we would need to use:

--Query to see who handled cash payments, how many & how much

```
SELECT
    ReceivedBy,
    COUNT(*) AS CashPayments,
    SUM(PaymentAmount) AS TotalCashHandled
FROM
    Payment
WHERE
    Type = 'Cash'
GROUP BY
    ReceivedBy
ORDER BY
    TotalCashHandled DESC;
```

My second suggestion is to provide an index for phone numbers to find the person's information. In my experience, the first thing I get asked when I call my pediatrician is "What is a good phone number?". Since phone numbers are unique, it provides a good way to search without providing duplicates, compared if one were to use last names instead. This provides a quick way to get a customer's information without sorting through a list of names that all could be similar. The type of indexing I would use would be hashing as it provides quick lookups based on equality tests and that is exactly what we need.

So for the SQL code...

Before the creation of the table we need to add:

```
DROP INDEX IF EXISTS idx_people_phonenumber;
```

After the creation of the tables we need to add:

```
CREATE INDEX idx_people_phonenumber ON People(PhoneNumber);
```

To query this we would need to do:

--Query to search people based on phone number, used '555-0123' from our ai generated data

```
SELECT *
FROM People
WHERE PhoneNumber = '555-0123';
```

6. Views

View 1: PatientPaymentOverview: shows how many payments a patient has made and the sum of all of their payments

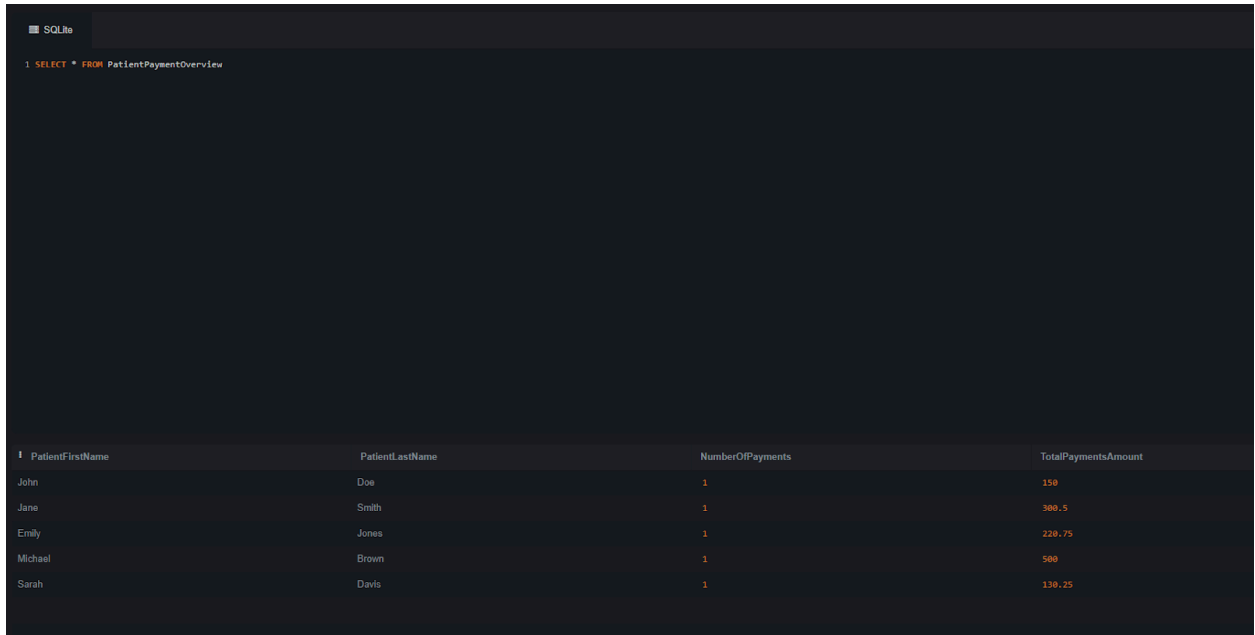
```
CREATE VIEW PatientPaymentOverview AS
SELECT
    p.FirstName AS PatientFirstName,
    p.LastName AS PatientLastName,
```

```

COUNT(pay.PaymentID) AS NumberOfPayments, -- count the number of payments made by
the patient
SUM(pay.PaymentAmount) AS TotalPaymentsAmount -- sum of all payment amounts made by
the patient
FROM Patients pt
JOIN People p ON pt.SSN = p.SSN -- join Patients with People to get first and last names
JOIN Payment pay ON pt.SSN = pay.SSN -- join Payment to count payments and sum the
amounts
GROUP BY p.SSN; -- group by SSN to get one row per patient

```

Output:



PatientFirstName	PatientLastName	NumberOfPayments	TotalPaymentsAmount
John	Doe	1	150
Jane	Smith	1	300.5
Emily	Jones	1	220.75
Michael	Brown	1	500
Sarah	Davis	1	130.25

View 2: All appointments in the last 30 days:

```

CREATE VIEW RecentAppointments AS
SELECT
p.FirstName AS PatientFirstName,
p.LastName AS PatientLastName,
a.TreatmentID,
t.StandardizedCost AS TreatmentCost,
a.AppointmentID,
a.Date AS AppointmentDate
FROM Appointments a
JOIN Patients pt ON a.SSN = pt.SSN
JOIN People p ON pt.SSN = p.SSN JOIN
Treatments t ON a.TreatmentID = t.TreatmentID -WHERE
a.Date >= date('now', '-30 day');

```

SQLite

```

1 SELECT * FROM RecentAppointments;
2

```

PatientFirstName	PatientLastName	TreatmentID	TreatmentCost	AppointmentID	AppointmentDate
John	Doe	1	120	101	2024-12-01
Jane	Smith	2	200	102	2024-12-02
Emily	Jones	1	120	103	2024-12-03
Michael	Brown	3	500	104	2024-12-04
Sarah	Davis	2	200	105	2024-12-05
Ran	Amamiya	1	120	111	2024-12-10
Ann	Takamaki	2	200	222	2024-12-11
Ryuji	Sakamoto	3	500	333	2024-12-12
Morgana	N/A	1	120	444	2024-12-13

7. Transactions

- Add a new medical staff member. Needs to be a single unit because if part of the transaction fails, there cannot be duplicate data, such as SSN within the database.

-- Step 1: Insert into People table

```
INSERT INTO People (SSN, FirstName, LastName, PhoneNumber)
VALUES ('555555555', 'James', 'Culver', '8357139507');
```

-- Step 2: Insert into MedicalStaff table

```
INSERT INTO MedicalStaff (SSN, MedType, Salary)
VALUES ('555555555', 'Dental Assistant', 50000);
```

-- Step 3: Insert into MedicalStaffHasLicense table

```
INSERT INTO MedicalStaffHasLicense (SSN, LicenseNumber)
VALUES ('555555555', 1); -- Replace '1' with a valid LicenseNumber
```

-- If no errors occur, commit the transaction

```
COMMIT;
```
- Add a new credit card payment and add it to the payments of a given bill, ensuring the payment is linked to an existing billing record. This also needs to be a single unit because of part of the transaction fails, there cannot be incomplete payments as it would ruin the integrity of the database and could lead to untraceable payments:

```
-- Start a new transaction
BEGIN TRANSACTION;
```

```
-- Your SQL commands here
INSERT INTO Payment (PaymentID, PaymentAmount, Type, ReceivedBy, SSN,
BillingRecordID)
VALUES (20, 350, 'Card', 'Jessica Culver', '123456789', 3);
```

```
INSERT INTO CardPayment (PaymentID, CardInfo)
VALUES (20, '12345678901234, 06/26, 538');
```

```
-- Commit the transaction
COMMIT;
```

8. Contributions & Final Thoughts

a.

Izaiah	#4, #6
Bladen	#1, #2, #3, #5
Sebastian	#7
Sai	#8

- b. Throughout the process of the project, our team has worked well! Looking back at the project we didn't have any major problems as we all were able to work smoothly. Although sometimes our schedules hadn't aligned, causing us to perform some work separately, as a team we were able to put together what we felt was quality work for each part of the project. Overall, the timeline was smooth sailing,